

SUPPLEMENTARY PROBLEM: SOLVING RECURRENCES

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Problem 1:

#	Recursion	Bound	MM Case
1	$T(n) = \frac{9}{4}T\left(\frac{2}{3}n\right) + n^2$		
2	$T(n) = 4T(n/2) + \sqrt{n}$		
3	$T(n) = 2T\left(n - \frac{n}{2} - \frac{n}{3}\right) + n$		
4	$T(n) = T(n/3) + T(2n/3) + \Theta(n)$		

Problem 1 Solutions

1. The recurrence equation follows the form solvable by Master's method. We can rewrite the recurrence such that it is easy to see a and b . $T(n) = \frac{9}{4}T\left(\frac{2}{3}n\right) + n^2 = \frac{9}{4}T\left(\frac{n}{3/2}\right) + n^2$. In this case, $a = 9/4$, $b = 3/2$, $f(n) = n^2$. From here, calculate the divide-function $g(n)$ and compare it to $f(n)$.

$$g(n) = n^{\log_b a} = n^{\log_{3/2}(9/4)} = n^2$$

Trivially, $f(n) = \Theta(n^2)$, therefore we have case 2 of Master Method, resulting in a runtime of

$$T(n) = \Theta(n^2 \lg n)$$

2. The recurrence equation follows the form solvable by Master's method. In this case $a = 4$, $b = 2$, $f(n) = n^{1/2}$. From here, calculate the divide-function $g(n)$ and compare it to $f(n)$.

$$g(n) = n^{\log_b a} = n^{\log_2(4)} = n^2$$

$\exists \epsilon = 1$ s.t. $f(n) = O(n^{2-\epsilon})$. Therefore, we use Case 1 of Master Method, resulting in a runtime of

$$T(n) = \Theta(n^2)$$

3. It may seem that at first we cannot use Master's Method. This would be true if the terms being subtracted are constants, but once the fraction is resolved we can write the recurrence as so

$$T(n) = 2T\left(n - \frac{n}{2} - \frac{n}{3}\right) + n = 2T(n/6) + n$$

where $a = 2$, $b = 6$, $f(n) = n$. Calculating the divide-function $g(n)$ and comparing it to $f(n)$ gives

$$g(n) = n^{\log_b a} = n^{\log_6(2)} < n$$

Since $\log_6(2) \approx 0.39$, we could choose $\epsilon = 0.1$ to say that $f(n) = \Omega(n^{0.39+\epsilon})$. Before we state Case 3 of Master Method, we must check the regularity condition.

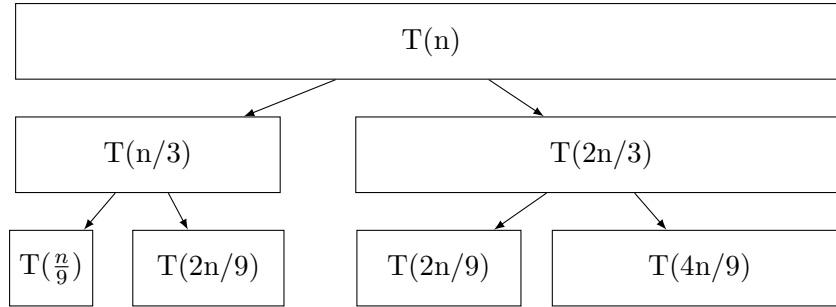
$$af\left(\frac{n}{b}\right) \leq cf(n)$$

$$2\left(\frac{n}{6}\right) \leq \frac{1}{2}n$$

Since we found a c s.t. $0 < c < 1$ that makes the inequality hold for all n , we therefore use Case 3 of Master Method to state the runtime as

$$T(n) = \Theta(n)$$

4. The recurrence does not follow the form of Master Method, so we must use recursion tree.



During the runtime calculation, we will focus on the right side of the tree since it splits at a slower rate. In order to calculate the max depth of the tree, we solve the following equation

$$n \left(\frac{2}{3}\right)^k = 1$$

$$\left(\frac{3}{2}\right)^k = n$$

$$k = \log_{3/2} n$$

Next, we must count the number of nodes at each level. To make the calculations easier, we'll assume that the number of nodes at level i of the tree is 2^{i-1} . Note that this is an upper bound on the number of nodes, since the right side of the tree will shallow out before the left side. Without this approximation, the problem would be much more complicated (even though it would result in the same runtime).

Usually, solving for the runtime of each level involves determining the number of nodes and multiplying the number of nodes by some constant amount of work. However, this gets quite complicated since the node sizes are not the same, and the number of nodes isn't always known at each level (since the right side of the tree is more shallow than the left). Therefore, we will make two useful assumptions:

- (a) the number of nodes at level i is 2^{i-1}
- (b) the size of all nodes at a level sum up to n

If we have some node of size n_1 , we then know that the work done for that node is $\Theta(n_1)$ (inferred from the recurrence equation). Therefore, assumption (b) allows us to say the work done per level of the tree is $\Theta(n)$.

Multiplying the number of levels by the work per level will give a final runtime of

$$T(n) = O(n \lg n)$$